

SmartLegal-AI

Detailed Master Tasklist + New Feature Roadmap

This version expands all 82 master tasks with what to change, why it matters, dependencies, implementation guidance and verification criteria.

P0 Task Backlog

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-001	Database schema authority What: Make Alembic the only production database schema authority. Remove/disable runtime CREATE TABLE logic so application startup cannot silently create or mutate production schema. Why: Prevents schema drift between local, staging and production. Every schema change becomes reviewable, reversible and reproducible. Dependencies: backend database, migrations Implementation: Create/normalize Alembic migrations; remove production create_tables() calls; verify fresh install and upgrade paths. Verify: Alembic upgrade/downgrade, clean database bootstrap
■	SL-002	Schema drift reconciliation What: Compare runtime schema, SQL migrations, Alembic migrations and actual model/query usage. Reconcile missing or conflicting columns such as file_type and any other drift discovered. Why: Different environments can otherwise expose different database structures and cause runtime failures. Dependencies: SL-001 Implementation: Generate a canonical schema; create migration for every missing/changed field; test upgrade from existing DB. Verify: Migration test + integration startup
■	SL-003	PostgreSQL-native data types What: Use UUID, TIMESTAMPTZ, JSONB and appropriate constraints where they improve correctness and queryability. Why: Native types reduce parsing, improve indexing and make data semantics explicit. Dependencies: SL-001 Implementation: Migrate compatible columns and update repository/schema code. Verify: Migration + CRUD integration tests
■	SL-004	Database indexes What: Add indexes for user/document/application ownership lookups, timestamps, analysis lookup, chat history and checklist retrieval. Why: History pages and joins will degrade as user data grows. Dependencies: SL-001 Implementation: Add targeted indexes based on actual query patterns; inspect query plans. Verify: EXPLAIN/query-plan verification
■	SL-005	Cascade and referential integrity What: Use foreign-key constraints and ON DELETE CASCADE selectively for dependent records such as document analyses/messages/checklists. Why: Prevents orphan records and reduces fragile manual deletion code. Dependencies: SL-002 Implementation: Audit every relationship and add safe cascades only where business semantics allow them. Verify: Deletion integration tests
■	SL-006	Generic service platform What: Replace separate Legal ID, Property and Business application architectures with one configurable Service/Application/Checklist platform. Why: These domains currently repeat the same CRUD, status and checklist behavior, creating unnecessary maintenance. Dependencies: SL-002 Implementation: Create service domain/type configuration and generic application models/services. Verify: All three domains pass existing workflows
■	SL-007	Shared ServiceHub frontend What: Create one reusable ServiceHub component driven by domain configuration instead of three near-identical hub pages. Why: Reduces duplicate UI and guarantees fixes propagate to every service domain. Dependencies: SL-006 Implementation: Implement generic props/config, migrate all three hubs, delete duplicated code. Verify: Frontend type-check + feature tests
■	SL-008	Shared ServiceDetail frontend What: Create one reusable ServiceDetail page for application details, status, checklist, notes and actions. Why: Property/Business/Legal ID detail pages share almost all structure. Dependencies: SL-006 Implementation: Move domain-specific differences into configuration and small extension components. Verify: All detail routes verified
■	SL-009	Generic backend application service What: Create shared application CRUD/business logic for create, list, get, update, delete and ownership checks. Why: Removes duplicated authorization and SQL logic from three routers. Dependencies: SL-006 Implementation: Introduce application service + repository and convert routers to thin HTTP adapters. Verify: API integration + authorization tests
■	SL-010	Generic checklist service What: Create one checklist engine supporting items, completion, required/optional state and application association. Why: Eliminates three checklist implementations and allows future services without new code. Dependencies: SL-006 Implementation: Create generic checklist schema/repository/service and configuration. Verify: Checklist tests across all domains
■	SL-011	Router/service/repository separation What: Keep FastAPI routers focused on HTTP validation/response mapping, move business rules into services and SQL into repositories. Why: Improves testability and prevents routers becoming large monolithic modules. Dependencies: SL-009 Implementation: Refactor incrementally; no business-rule duplication in routers. Verify: Unit tests for services + integration tests

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-012	Durable analysis workers What: Replace FastAPI BackgroundTasks for long-running AI processing with a persistent Redis-backed queue/worker system. Why: In-process background jobs can disappear during restarts/deployments. Dependencies: Redis infrastructure, SL-001 Implementation: Use a worker framework such as ARQ; persist job state and retry safely. Verify: Worker restart/retry integration test
■	SL-013	Private document storage What: Store legal documents privately and issue short-lived signed download URLs only after ownership authorization. Why: Public document URLs can expose highly sensitive legal data. Dependencies: Auth, object storage Implementation: Store object keys rather than public URLs; authorize download before signing. Verify: Cross-user access/security tests
■	SL-014	Secure session/token storage What: Move browser authentication away from localStorage JWT storage toward HttpOnly, Secure, SameSite cookies or an equally hardened session design. Why: Reduces token theft impact from XSS and makes session handling more secure. Dependencies: Auth API Implementation: Implement cookie auth, CSRF strategy and logout/revocation behavior. Verify: Auth E2E + security tests
■	SL-015	Upload hardening What: Stream uploads, enforce size/page limits before buffering, validate magic bytes/content type, and reject malformed or dangerous files. Why: Protects memory, storage and processing resources from malicious or accidental oversized input. Dependencies: SL-013 Implementation: Implement streaming validation and explicit document limits. Verify: Oversized/fake-file security tests
■	SL-016	AI abuse controls What: Add per-user/IP rate limits, quotas and concurrency limits for uploads, analysis, chat and expensive AI operations. Why: Prevents accidental or malicious AI-cost and resource explosions. Dependencies: SL-012 Implementation: Add Redis-backed rate limiting and usage accounting. Verify: 429/rate-limit tests
■	SL-017	Prompt-injection boundaries What: Treat all uploaded document text as untrusted data and explicitly prevent document content from overriding system instructions. Why: Legal documents can contain adversarial or instruction-like text. Dependencies: AI pipeline Implementation: Add strict prompt boundaries, structured inputs and adversarial test cases. Verify: Prompt-injection regression suite

P1 Task Backlog

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-018	AI provider abstraction What: Separate provider-independent AI orchestration from Groq/Gemini-specific API code. Why: Makes provider changes, fallback and testing easier. Dependencies: SL-012 Implementation: Create AI core/router plus provider adapters. Verify: Provider mock tests
■	SL-019	Groq/Gemini fallback What: Keep provider adapters small and implement controlled fallback only where appropriate. Why: Improves resilience when a primary provider is unavailable. Dependencies: SL-018 Implementation: Define timeout/retry/fallback policy and prevent duplicate expensive requests. Verify: Provider failure integration tests
■	SL-020	Centralized AI prompts What: Move prompts into capability-specific modules for classification, analysis, summary, chat and negotiation. Why: Makes prompt versions reviewable and avoids prompt duplication. Dependencies: SL-018 Implementation: Create prompt registry with explicit versions. Verify: Prompt snapshot/regression tests
■	SL-021	AI parsing/validation layer What: Centralize structured-output parsing, Pydantic validation, malformed JSON repair and provider error handling. Why: Every AI feature should not implement its own JSON parsing. Dependencies: SL-018 Implementation: Create shared parser and typed result schemas. Verify: Malformed-output tests
■	SL-022	Persist extraction What: Persist extracted text and processing artifacts so chat/analysis does not repeatedly parse the original document. Why: Reduces latency and CPU usage. Dependencies: SL-012 Implementation: Add document_processing/artifact storage and retrieval. Verify: Repeated-analysis benchmark
■	SL-023	Document hashing What: Calculate SHA-256 for uploaded files and use it for deduplication and processing reuse. Why: Avoids duplicate extraction and AI cost for identical files. Dependencies: SL-022 Implementation: Store content hash and enforce safe reuse rules. Verify: Duplicate-upload tests

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-024	Analysis versioning What: Persist model, prompt version, KB version, pipeline version and document hash with each analysis. Why: Makes results reproducible and prevents stale cache reuse after pipeline changes. Dependencies: SL-020, SL-023 Implementation: Create analysis metadata and version-aware cache keys. Verify: Version-change cache tests
■	SL-025	Durable analysis jobs What: Track queued/extracting/OCR/classifying/analyzing/validating/completed/failed stages with retries and progress. Why: Enables reliable processing, progress UI and operational debugging. Dependencies: SL-012 Implementation: Create analysis_jobs table and worker state transitions. Verify: Worker lifecycle tests
■	SL-026	Bounded chunk concurrency What: Process analysis chunks concurrently with a strict semaphore rather than fully sequential or unbounded parallel execution. Why: Reduces latency while respecting provider limits. Dependencies: SL-018 Implementation: Add configurable concurrency, backoff and cancellation. Verify: Load/rate-limit benchmark
■	SL-027	OCR/page-aware extraction What: Add robust OCR fallback, page boundaries and source-page metadata. Why: Scanned PDFs and evidence citations require page-aware text. Dependencies: SL-022 Implementation: Persist page/section metadata during extraction. Verify: Scanned-document tests
■	SL-028	Verified legal references What: Replace free-form legal citations with IDs resolved against verified legal-source records. Why: LLMs can hallucinate sections or cite outdated law. Dependencies: Legal knowledge model Implementation: Create legal_reference records with act/section/source/effective dates. Verify: Citation verification tests
■	SL-029	Evidence/confidence model What: Store evidence snippets, source pages, confidence and uncertainty for AI findings. Why: Users need to understand why a risk was identified and how reliable it is. Dependencies: SL-027, SL-028 Implementation: Extend analysis schema and UI contracts. Verify: Schema + UI tests
■	SL-030	Reusable legal retrieval What: Build a retrieval interface that can later support PostgreSQL full-text search/pgvector without changing feature code. Why: Creates a stable foundation for grounded legal AI. Dependencies: SL-028 Implementation: Define retrieval interface, metadata filters and ranking contract. Verify: Retrieval unit tests
■	SL-031	Split Compare page What: Break the oversized Compare.tsx into search, filters, article list/cards, modal, overview, clauses, checklist and resources components. Why: Large monolithic components are difficult to test and modify. Dependencies: Frontend architecture Implementation: Extract focused components without changing UX. Verify: Type-check + component tests
■	SL-032	Split Analysis page What: Break Analysis.tsx into dashboard, summary, risks, clauses, evidence, export and processing-state components. Why: Makes the core analysis experience maintainable and extensible. Dependencies: SL-029 Implementation: Extract components and feature hooks. Verify: Type-check + UI tests
■	SL-033	Correct Compare/Knowledge Base separation What: Align routes, filenames and navigation so document comparison is distinct from Knowledge Base functionality. Why: Current naming creates product and code confusion. Dependencies: SL-031 Implementation: Use /compare for comparison and /knowledge-base for KB, or clearly separate tabs. Verify: Route/E2E tests
■	SL-034	Complete document comparison What: Implement the missing comparison UI around the existing comparison state/API foundations. Why: A major backend capability exists without a complete user workflow. Dependencies: SL-033, SL-024 Implementation: Build upload, processing, results and comparison views. Verify: Comparison E2E test
■	SL-035	Feature-based frontend What: Organize frontend code by product domain: auth, documents, analysis, comparison, chat, advisor, insights, services and knowledge-base. Why: Makes ownership and dependency boundaries clearer. Dependencies: SL-007 Implementation: Move incrementally and preserve imports. Verify: Build/type-check
■	SL-036	Typed API layer What: Create typed API modules for auth, documents, analysis, chat, advisor and services behind one HTTP client. Why: Removes scattered URLs, response parsing and error handling. Dependencies: Frontend architecture Implementation: Centralize request/response types and API methods. Verify: API mock tests

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-037	RTK Query/server-state reduction What: Evaluate and migrate repetitive API thunks to RTK Query where appropriate; keep Redux for genuine client state. Why: Removes loading/error/cache/refetch boilerplate. Dependencies: SL-036 Implementation: Migrate feature by feature and delete obsolete thunks. Verify: Regression tests
■	SL-038	Shared UI primitives What: Create reusable Modal, Toast, ConfirmDialog, EmptyState, Skeleton, StatusBadge and form primitives. Why: Eliminates repeated UI behavior and improves consistency/accessibility. Dependencies: Design system Implementation: Create accessible primitives and migrate existing pages. Verify: Component tests
■	SL-039	Shared configuration/helpers What: Centralize status/risk configs, file-size/date formatting, API errors, validation and common feature constants. Why: Prevents small duplicated rules from diverging. Dependencies: SL-036 Implementation: Create shared lib/config modules and replace copies. Verify: Unit tests
■	SL-040	Notification system What: Replace browser alert() usage with accessible toast/notification and confirmation patterns. Why: Improves UX, keyboard accessibility and consistency. Dependencies: SL-038 Implementation: Add toast provider and error/success patterns. Verify: UI tests
■	SL-041	Frontend performance What: Lazy-load routes and heavy features, dynamically import expensive PDF/export tooling, and avoid shipping large KB data unnecessarily. Why: Improves initial load and runtime performance. Dependencies: SL-035 Implementation: Measure bundle before/after and split heavy modules. Verify: Production build + bundle analysis
■	SL-042	Design system/tokens What: Create semantic design tokens for surfaces, text, primary actions, risk levels, success/warning/error and spacing/typography. Why: Reduces visual inconsistency and makes redesigns cheap. Dependencies: SL-038 Implementation: Replace scattered hardcoded styles incrementally. Verify: Visual review
■	SL-043	Accessibility pass What: Fix focus management, keyboard navigation, aria labels, contrast, loading announcements and semantic controls. Why: Makes the application usable with keyboard and assistive technologies. Dependencies: SL-038 Implementation: Audit major workflows and remediate issues. Verify: Accessibility audit + manual keyboard test
■	SL-044	Backend pytest suite What: Create unit/integration test structure for auth, documents, analysis, chat, services and legal retrieval. Why: Enables safe refactoring and prevents regressions. Dependencies: Architecture refactor Implementation: Use fixtures, isolated test DB and mocked external providers. Verify: pytest in CI
■	SL-045	Authorization/IDOR tests What: Verify users cannot read/update/delete another user's documents, analyses, chats or applications. Why: Object-level authorization is critical for private legal data. Dependencies: SL-013, SL-014 Implementation: Create explicit negative tests for every endpoint. Verify: Security test suite
■	SL-046	Upload/rate-limit security tests What: Test oversized files, invalid content, repeated requests, concurrency and quota exhaustion. Why: Prevents common abuse paths from returning during future refactors. Dependencies: SL-015, SL-016 Implementation: Automate attack/regression cases. Verify: Security CI
■	SL-047	Frontend tests What: Add component/feature tests for auth, upload, analysis, comparison, chat and service tracking. Why: Protects critical user flows. Dependencies: SL-035 Implementation: Prioritize business-critical paths. Verify: Test runner in CI
■	SL-048	Playwright E2E What: Add browser tests covering register/login, upload, analysis, chat, comparison and service tracking. Why: Catches frontend/backend integration failures. Dependencies: SL-047 Implementation: Use deterministic fixtures/mocks where external AI is unsuitable. Verify: Playwright CI
■	SL-049	CI quality gates What: Run TypeScript, build, ESLint, pytest, E2E/security checks and migration validation in CI. Why: Prevents broken code from reaching main. Dependencies: SL-044 Implementation: Create pipeline with staged fast/slow checks. Verify: CI green

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-050	ESLint configuration What: Add a working ESLint configuration matching the installed toolchain and enforce it in CI. Why: The project currently has an ESLint command without a complete configuration. Dependencies: SL-049 Implementation: Configure React/TypeScript rules and fix actionable violations. Verify: npm lint
■	SL-051	Dependency cleanup What: Remove unused frontend/backend dependencies and dead SDKs after verifying actual imports/runtime use. Why: Reduces attack surface, install size and confusion. Dependencies: Architecture cleanup Implementation: Generate usage inventory and remove safely. Verify: Clean install/build/tests
■	SL-052	Repository hygiene What: Remove generated dependencies, logs, databases, virtual environments, build output and unnecessary agent/editor directories from source control. Why: Keeps repository small and reproducible. Dependencies: Architecture cleanup Implementation: Update .gitignore and purge tracked artifacts. Verify: Clean clone verification
■	SL-053	Environment/deployment docs What: Document environment variables, local setup, migrations, workers, storage, Redis and production deployment. Why: Reduces setup errors and operational dependency on tribal knowledge. Dependencies: SL-001, SL-012 Implementation: Create current deployment/runbook docs. Verify: Fresh-machine setup test
■	SL-054	Architecture documentation sync What: Keep architecture docs aligned with actual source, routes, DB, workers and AI provider behavior. Why: Stale documentation misleads future AI agents and developers. Dependencies: All architecture tasks Implementation: Add documentation update to architecture PR definition of done. Verify: Review against source

P2 Task Backlog

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-055	Password reset completion What: Implement real password-reset email delivery, secure reset URL, rate limiting, token cleanup and session revocation. Why: The current development flow does not constitute a complete production reset feature. Dependencies: Auth provider/email service Implementation: Integrate email provider and production-safe reset flow. Verify: E2E reset test
■	SL-056	Analysis progress UI What: Display real processing stages and progress from analysis_jobs rather than generic loading state. Why: Users need feedback during long AI processing. Dependencies: SL-025 Implementation: Add polling/WebSocket/SSE strategy and stage UI. Verify: E2E processing state
■	SL-057	Pagination What: Add cursor/page pagination for document history, chats and application lists. Why: Prevents large responses and slow rendering as data grows. Dependencies: SL-004 Implementation: Add API pagination contracts and reusable frontend pagination. Verify: Large-fixture performance test
■	SL-058	Application timeline What: Record and display status changes, submissions, notes and important events for each service application. Why: Turns a basic tracker into a useful workflow history. Dependencies: SL-006 Implementation: Create application_events model and timeline UI. Verify: Timeline tests
■	SL-059	Reminders and important dates What: Allow users to track deadlines, renewal dates and required actions. Why: Legal workflows are time-sensitive and users benefit from proactive reminders. Dependencies: SL-058 Implementation: Create reminders model and scheduler. Verify: Reminder scheduling tests
■	SL-060	Obligation extraction What: Extract user obligations, deadlines, payment duties and required actions from analyzed documents. Why: Transforms analysis into actionable tasks. Dependencies: SL-029 Implementation: Extend AI schema and create obligation records. Verify: Extraction fixtures
■	SL-061	Renewal/expiry alerts What: Detect renewal, expiry and notice periods and alert users before deadlines. Why: Prevents missed contractual deadlines. Dependencies: SL-059, SL-060 Implementation: Create reminder policy and notification delivery. Verify: Date-boundary tests

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-062	Cross-document AI What: Allow users to ask questions across multiple owned documents with evidence and source-document attribution. Why: Enables high-value questions such as comparing old/new agreements. Dependencies: SL-030, SL-057 Implementation: Build retrieval context across selected documents. Verify: Cross-document E2E
■	SL-063	Document version history What: Track document versions and explain what changed between uploads. Why: Useful for contracts undergoing negotiation. Dependencies: SL-023 Implementation: Link versions by document family and hash. Verify: Versioning tests
■	SL-064	Clause-level diffing What: Normalize documents into clauses and show added/removed/changed clauses with risk impact. Why: Much more useful than raw text diff for legal documents. Dependencies: SL-034, SL-063 Implementation: Create clause normalization and comparison result schema. Verify: Comparison fixtures
■	SL-065	Contract Health Score What: Provide an overall score plus financial, termination, liability, privacy, dispute and user-protection dimensions. Why: Gives users a quick actionable overview. Dependencies: SL-029 Implementation: Define deterministic scoring rubric and map validated AI findings into it. Verify: Scoring unit tests
■	SL-066	Before You Sign What: Create a focused workflow answering what the user agrees to, costs, termination rights, obligations, consequences and negotiation points. Why: Provides a clear consumer-facing decision workflow. Dependencies: SL-029, SL-065 Implementation: Create dedicated analysis prompt/schema/UI with evidence. Verify: E2E acceptance tests
■	SL-067	Clause rewrite/negotiation What: Suggest safer alternative wording while clearly distinguishing generated suggestions from legal advice. Why: Turns risk detection into an actionable negotiation tool. Dependencies: SL-029, SL-066 Implementation: Generate alternatives with rationale and evidence; add disclaimer/escalation. Verify: Prompt regression tests
■	SL-068	Lawyer escalation/referral What: Provide a controlled path from AI analysis to human legal review when risk/confidence/stakes justify it. Why: Adds a safety net for high-stakes cases. Dependencies: SL-029 Implementation: Define referral workflow, consent and data-sharing permissions. Verify: Permission/privacy tests
■	SL-069	Consistent legal disclaimer What: Create reusable legal-information disclaimer and escalation guidance across AI features. Why: Sets correct user expectations and reduces ambiguous AI advice presentation. Dependencies: SL-029 Implementation: Centralize component/content and apply to all relevant views. Verify: UI audit

P3 Task Backlog

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-070	Regional-language support What: Add India-focused UI and AI output support for Marathi, Hindi, Tamil, Telugu, Bengali and additional languages. Why: Expands accessibility and market reach. Dependencies: Frontend i18n, AI abstraction Implementation: Introduce translation catalogs and language-aware AI schemas. Verify: i18n/E2E tests
■	SL-071	Voice interaction What: Add voice input/output for appropriate legal questions and document workflows. Why: Improves accessibility and hands-free use. Dependencies: Regional languages, privacy controls Implementation: Add browser/device-compatible speech layer with clear consent. Verify: Voice UX/security review
■	SL-072	WhatsApp assistant What: Allow users to ask document/legal questions and receive reminders through WhatsApp. Why: High potential distribution channel in India. Dependencies: Auth, notification engine, privacy Implementation: Implement secure identity linking and limited message flows. Verify: Integration/security tests
■	SL-073	PWA/mobile experience What: Create responsive PWA/mobile-first experience after API and feature architecture stabilizes. Why: Improves mobile accessibility without prematurely duplicating codebases. Dependencies: Frontend architecture Implementation: Use responsive feature components and installable PWA shell. Verify: Mobile E2E

Status	ID / Change	Detailed change / why / implementation / verification
■	SL-074	Subscriptions/quotas/billing What: Add usage plans, AI quotas, storage quotas and subscription billing. Why: Creates sustainable monetization and protects expensive AI resources. Dependencies: SL-016 Implementation: Implement usage ledger and payment provider abstraction. Verify: Billing sandbox tests
■	SL-075	Organization workspaces What: Support teams with shared documents, roles and organization-level quotas. Why: Enables professional/team use. Dependencies: SL-068, permissions Implementation: Add organization/member/role model and permission matrix. Verify: Authorization matrix tests
■	SL-076	Granular document sharing What: Allow controlled read-only or collaborative sharing with revocation and expiry. Why: Useful for lawyers, family members and teams. Dependencies: SL-075, SL-013 Implementation: Create share grants with scope/expiry. Verify: Security tests
■	SL-077	Admin knowledge management What: Create admin tooling for service guidance, legal sources, FAQs, fees, timelines and official links. Why: Allows updates without code deployments. Dependencies: SL-028, dynamic catalog Implementation: Add versioned content models and admin authorization. Verify: Admin authorization tests
■	SL-078	Audit/security events What: Record security-sensitive actions such as login, document access, sharing, deletion and permission changes. Why: Improves incident response and compliance readiness. Dependencies: SL-014, SL-076 Implementation: Create append-only audit event model with retention policy. Verify: Audit event tests
■	SL-079	Official-source ingestion What: Ingest and version official legal/service sources with provenance and effective dates. Why: Keeps legal guidance current and auditable. Dependencies: SL-028, SL-077 Implementation: Build ingestion/validation pipeline and source metadata. Verify: Source/version tests
■	SL-080	Hybrid legal retrieval What: Add PostgreSQL full-text + pgvector retrieval when the legal corpus warrants semantic search. Why: Improves grounded retrieval without prematurely adding another database. Dependencies: SL-030, SL-079 Implementation: Benchmark lexical/semantic/hybrid ranking. Verify: Retrieval benchmark
■	SL-081	AI operations dashboard What: Track AI cost, latency, token usage, failures, retries, provider health and feature usage. Why: Makes AI operations measurable and controllable. Dependencies: SL-012, SL-018 Implementation: Emit structured metrics and build internal dashboard. Verify: Metric validation
■	SL-082	Product analytics What: Track feature adoption, document processing funnel, retention and failure points without exposing document contents. Why: Helps prioritize improvements using real usage. Dependencies: Privacy model Implementation: Define privacy-safe events and retention policy. Verify: Analytics privacy review

New Feature Addition Roadmap

Document Intelligence

- **F-001 — Document Comparison** [P0] — Compare two agreements at clause level, identify changed/added/removed clauses, show risk and evidence. Dependencies: SL-034, SL-064
- **F-002 — Before You Sign** [P0] — Provide a guided summary of commitments, costs, rights, termination, obligations, consequences and negotiation points. Dependencies: SL-066
- **F-003 — Contract Health Score** [P0] — Show overall and category-level risk scores with evidence-backed explanations. Dependencies: SL-065
- **F-004 — Document Version History** [P1] — Maintain versions of a document family and summarize changes between versions. Dependencies: SL-063
- **F-005 — Cross-Document AI** [P1] — Answer questions across selected documents with source-document and page attribution. Dependencies: SL-062
- **F-006 — Obligation Tracker** [P1] — Turn extracted obligations and deadlines into actionable user tasks. Dependencies: SL-060
- **F-007 — Renewal & Expiry Alerts** [P1] — Detect contractual dates and proactively remind users before deadlines. Dependencies: SL-061
- **F-008 — Red-Flag Scanner** [P1] — Provide a fast first-pass scan of unusual, one-sided or high-risk clauses. Dependencies: SL-029, SL-065

Legal Trust & Safety

- **F-009 — Verified Legal Citations** [P0] — Resolve AI citations against a versioned legal-source database. Dependencies: SL-028
- **F-010 — Evidence & Page Citations** [P0] — Show the exact document evidence and page supporting an AI finding. Dependencies: SL-029
- **F-011 — Confidence & Uncertainty** [P1] — Clearly indicate when the model is uncertain or evidence is insufficient. Dependencies: SL-029
- **F-012 — Human Lawyer Escalation** [P1] — Offer human review for high-stakes or low-confidence cases. Dependencies: SL-068
- **F-013 — Official Source Updates** [P1] — Keep legal/service knowledge synchronized with authoritative sources. Dependencies: SL-079

Service Platform

- **F-014 — Dynamic Service Catalog** [P1] — Store requirements, fees, timelines, authority and official links as versioned data. Dependencies: SL-006, SL-077
- **F-015 — Application Timeline** [P1] — Show status history and important events for every service application. Dependencies: SL-058
- **F-016 — Smart Checklist** [P1] — Generate and track required/optional documents for each service. Dependencies: SL-010
- **F-017 — Deadline Reminders** [P1] — Remind users about missing documents and application deadlines. Dependencies: SL-059

Productivity

- **F-018 — Clause Library** [P1] — Save, tag and reuse favorable or negotiated clauses. Dependencies: SL-063
- **F-019 — Folders & Tags** [P2] — Organize documents and analyses without changing document ownership/security. Dependencies: SL-057
- **F-020 — Bulk Processing** [P2] — Upload/process multiple documents under controlled concurrency and quotas. Dependencies: SL-026, SL-016
- **F-021 — Shareable Reports** [P2] — Generate read-only reports with controlled access and expiry. Dependencies: SL-076

India-focused Expansion

- **F-022 — Regional Languages** [P2] — Localize UI and AI output for major Indian languages. Dependencies: SL-070
- **F-023 — WhatsApp Assistant** [P2] — Provide document questions and reminders through WhatsApp. Dependencies: SL-072
- **F-024 — Voice Legal Assistant** [P3] — Enable voice questions and answers with consent and language support. Dependencies: SL-071
- **F-025 — Region-Aware Guidance** [P2] — Adapt service guidance to state/region through configuration rather than duplicated UI code. Dependencies: SL-014, SL-077

Monetization & Professional

- **F-026 — Usage Plans** [P2] — Meter AI, storage and processing usage by account/plan. Dependencies: SL-074
- **F-027 — Team Workspaces** [P2] — Support organizations, roles, shared documents and team quotas. Dependencies: SL-075
- **F-028 — Paid Human Review** [P3] — Allow users to purchase professional review after AI analysis. Dependencies: SL-068, SL-074
- **F-029 — Enterprise Audit** [P3] — Provide stronger audit/compliance controls for professional customers. Dependencies: SL-078

Agent status: ☐ Not Started • ☐ In Progress • ☒ Complete/verified • ☐ Blocked • ☐ Needs Review